

先出 Console 版，确认日志真的落地，再出 --windowed

Phase 1 唯一碰启动路径的东西是新的诊断日志层。它在 macOS 上全部验证过，但 Windows 分支从没在 Windows 上跑过。这份清单就是去踩那几个分支。

先说清楚这份清单的性质。下面每条「该看到什么」都是预期值，来自代码和 macOS 上的实测，不是 Windows 上的实测——因为构建必须在 Windows 上跑，PyInstaller 不能交叉编译。你做的正是这件验证工作，对不上就是发现问题。

构建机要求

Windows + Python 3 (`py -3` 能解析到)

通过条件

C-01 至 C-06 全绿

代码基线

`a5ef318` 已提交，无未提交 .py

失败即停

任一条不符 → 别出 windowed 版

PART 1 — 六步验收

顺序不能换：Console 版是唯一能看见启动错误的形态

每步三段：做什么 / 该看到什么 / 看不到怎么办。C-01 到 C-04 用 Console 版，C-05 才切 windowed。

C-01 出 Console 版构建

做什么

■ 做什么

```
cd <仓库根目录>
tools\build_windows_folder.bat -Console
```

首次构建会建 `.venv-build-win` 并装依赖，比较慢。

■ 该看到什么

脚本跑完不抛错，并且中途出现依赖契约校验步骤：

```
==> Verifying frozen import dependency contract
...
产物: dist\TraceLab7.9\TraceLab7.9.exe
```

■ 看不到怎么办

报「requirements.txt 缺少 ...」→ 依赖清单和冻结导入不一致，不是 Phase 1 引入的，按提示补。

报 PowerShell 执行策略 → 用 `.bat` 包装器（它已带 `-ExecutionPolicy Bypass`），别直接调 `.ps1`。

C-02 启动 exe，看它活不活

■ 做什么

```
dist\TraceLab7.9\TraceLab7.9.exe
```

别用 `tools\run_windows_exe.bat` 不带参数——它里面写死的默认名还是 `TraceLab7.6`，和构建脚本的默认 `7.9` 对不上，会报「exe not found」。要用就传参：`tools\run_windows_exe.bat TraceLab7.9`。

■ 该看到什么

主窗口正常出现，控制台窗口没有 traceback。特别注意这一条不该出现：

```
AttributeError: module 'logging' has no attribute 'getLevelNamesMap'
```

这是 followup Task 1 修掉的那个 3.11-only API。如果它出现了，说明改动没进这个构建。

■ 看不到怎么办

窗口没出来但控制台有 **traceback** → 直接把 traceback 贴出来，这正是 Console 版存在的意义。

出现上面那条 **AttributeError** → 构建用的是旧代码。确认 `git log -1` 是 `a5ef318` 或更新，然后加 `-SkipInstall:$false` 重建。

C-03 日志文件真的建出来了吗

■ 做什么

应用还开着的时候，另开一个 cmd：

```
dir "%LOCALAPPDATA%\TraceLab\logs"
type "%LOCALAPPDATA%\TraceLab\logs\tracelab.log"
```

■ 该看到什么

`tracelab.log` 存在，且第一行是这个形状：

```
2026-07-31 09:12:03,441 INFO [MainThread] mf4_analyzer.diagnostics:
```

这一行同时证明三件事：目录解析对了、目录可写、handler 装上了。

■ 看不到怎么办

目录不存在 → 路径解析或权限问题。用 `set TRACELAB_LOG_DIR=D:\tracelab-logs` 再启一次，能写说明是 `%LOCALAPPDATA%` 权限，不能写说明是 handler 本身。

目录在但文件是空的 → handler 建了但没刷。应用正常退出后再看一次。

应用能跑但没有日志 → 这是设计内的降级（诊断失败绝不阻止启动）。控制台里应该有 `Diagnostics file is unavailable`

应该有 `Diagnostics file is unavailable` 。

C-04 日志内容对不对（噪声过滤 + Qt 桥接）

■ 做什么

在应用里导入一个 MF4 文件、画几个通道、拖拽缩放几下，然后再看日志。

■ 该看到什么

不该出现第三方库的 INFO 噪声（followup Task 3 过滤掉了）：

```
× 不该有 INFO ... numexpr.utils: NumExpr defaulting to 10 threads.
× 不该有 INFO ... asammf.blocks...: <任何 INFO 级噪声>

✓ 可以有 WARNING ... mf4_analyzer.diagnostics: Qt message category=.
✓ 可以有 INFO ... mf4_analyzer.<任何自家模块>: ...
```

规则是：自家 `mf4_analyzer.*` 的 INFO 保留，第三方只留 WARNING 及以上。

■ 看不到怎么办

满屏第三方 INFO → 过滤器没生效，构建是旧代码，回 C-02 确认版本。

日志完全是空的（只有第一行） → 正常。没有异常就没有记录，这是好消息，不是故障。

C-05 切 windowed 版重建

■ 做什么

```
tools\build_windows_folder.bat
```

不带 `-Console` 就是 `--windowed`，即 GUI 子系统、无控制台。

■ 该看到什么

构建成功，且脚本末尾的打包冒烟通过，产出这个文件：

```
docs\analyzer\acquisition\evidence\vector-xcp\packaged-runtime-smoke  
{"ok": true, ...}
```

■ 看不到怎么办

冒烟报 `ok: false` → 脚本会直接 throw 并指向那个 json。先读它，多半是采集侧的 Vector/XCP 运行时，跟 Phase 1 无关。

C-06 windowed 版的关键分支：没有 stderr 也要能写日志

■ 做什么

删掉旧日志，跑 windowed 版，做和 C-04 一样的操作，再看日志。

```
del "%LOCALAPPDATA%\TraceLab\logs\tracelab.log"  
dist\TraceLab7.9\TraceLab7.9.exe
```

■ 该看到什么

日志照样被重新建出来，第一行还是 `Diagnostics initialized`。

这是整份清单最重要的一条。windowed 构建下 `sys.stderr` 可能是 `None`，也可能是一个「写就抛异常」的对象。代码判的是 `is not None` —— 前者处理了，后者是从没跑过的分支。这一步就是去踩它。

■ 看不到怎么办

windowed 版没日志，但 Console 版有 → 抓到了，就是 stderr 分支。现象是应用可能直接静默退不出来或没窗口。这时把 `set TRACELAB_LOG_DIR=...` 加上再试一次可以确认是不是 handler 的问题。

应用根本不启动，没有任何输出 → 这就是 windowed 版的典型形态。回退到 Console 版看 traceback，别在 windowed 上猜。

正常使用体现不出来，这是设计如此

Phase 1 里六个包，只有一个会在正常使用中产出可观察的东西。别指望「用起来感觉变稳了」——那不是这轮的产出。

工作包	正常使用感觉得到吗	怎么确认
W1a 同名通道 Y 轴串台	只有踩到的人	主动复现，见下方折叠区
W1b moving_avg 长度	只有踩到的人	主动复现，见下方折叠区
W1c _ChannelKeyDict	不会	拆的是地雷，当前无活调用点。价值在未来重构不会重踩
W2 诊断日志	会	唯一一个。形式是「以前看不见的现在能看见」，不是「变快变稳」
W3 状态卫生	不会	零行为变化就是它的验收标准
W4 测试分类研究	不会	没改代码，产出是一份报告

所以真正的判据只有一条：正常用两三周，然后去读 `tracelab.log`。三种结果，每一种都有明确含义和明确的下一步。

结果 A

日志基本是空的

只有启动那行 `Diagnostics`

结果 B

反复出现某几条

同一个 `channel` `key=` 或同一个 `seam`

initialized，几周下来没积累出东西。

含义：那 288 处宽泛静默 except 大部分确实是良性的，审计对风险的估计偏高。

下一步：「收窄异常处理」这项取消。这是好结果，不是白干——你花两周买到了「不用改那 288 处」这个结论。

名字一遍遍出现，可能带 suppressed N occurrences。

含义：找到真实故障了。这些就是一一直在静默降级、但从来没人知道的路径。

下一步：这份日志就是下一轮的修复清单——按频次排序，逐条查。这是 W2 存在的全部意义。

结果 C

日志爆量 / 文件反复轮转

几天就把 5 MiB × 5 的配额跑满，或者出现巨大的 suppressed 计数。

含义：要么限频窗口对你的使用模式不合适，要么真有系统性故障在高频触发。

下一步：先看是哪个 key 在刷屏。如果是单一 key，那是个真 bug；如果是很多 key 各刷一点，调 WINDOW / BURST。

► 想现在就亲眼看到两个 bug 修好了？这样复现

脚注

两件这份清单管不到的事

- 测试套件目前当不了门禁。完整 pytest 在本机 51% 处 SIGSEGV，崩在 db_reference_dialog.py:117 的 Qt 对象生命周期上——这是 Phase 1 之前就有的，不是这轮引入的。在它修好之前，「所有测试绿」这句话没人能说。

- **Phase 2 不用按原计划做。** W4 已经把它一半内容划掉了（whitebox 规则取消；`canvas.py` 不是被测试阻塞，A 类只有 3.6%）。剩下最有价值的那项——收窄异常处理——门控恰恰是上面结果 A/B/C 这份日志。**先用起来，再谈下一轮。**

